

Software Design Description (SDD)

Project: Undergraduate Transfer Management System (UTMS)

Team: Team 3

- Egehan Vardar 323011010
- Mehmet Fatih Gülmez 323011006

Date: December 17, 2025

Standard: IEEE STD 1016

Version	Date	Description of Changes	Author
1.0	2025-12-17	Initial draft encompassing Logical, Information, and Interaction Viewpoints ¹⁰ .	Team 3
1.1	2025-12-21	Integration of Interface Viewpoint, Glossary, and References per standard requirements.	Team 3
1.2	2025-12-21	Formatting for full IEEE STD 1016 conformance including Fronts Piece and Introduction.	Team 3

1. Introduction.....	4
2. References.....	5
3. Glossary.....	6
4. Identified Stakeholders and Design Concerns.....	7
5. Logical Viewpoint.....	8
5.1 Subsystem Decomposition.....	8
5.2 Design Class Diagram.....	9
6. Information Viewpoint.....	10
6.1 Logical Database Schema (ERD).....	10
6.2 Data Attribute Dictionary.....	11
Application Entity:.....	11
Document Entity:.....	11
7. Interaction Viewpoint.....	12
7.1 Interaction View 1: Student Application Submission.....	12
7.2 Interaction View 2: YGK Application Evaluation.....	13
7.3 Interaction View 3: Administrative Validation and Routing.....	14
7.4 Interaction View 4: Final Approval and Publication:.....	15
7.5 Interaction View 5: Application Resubmission:.....	16
7.6 Interaction View 6: Secure Password Reset.....	17
7.7 Interaction View 7: Admin Rule Configuration.....	18
8 Design Rationale.....	19
8.2 Data Strategy: Protecting the Academic Record.....	19
8.3 Data Strategy: Protecting the Academic Record:.....	19
8.4 Centralized Logic: Ensuring Rule Consistency.....	20
8.5 Security: Building Trust Through Accountability.....	20

1. Introduction

1.1 Purpose

The purpose of this **Software Design Description (SDD)** is to specify the information content and organization required to implement the **Undergraduate Transfer Management System (UTMS)**. It provides the technical and managerial stakeholders with an explicit design description that satisfies the requirements outlined in the SRS.

1.2 Scope

This document identifies the design views, viewpoints, and entities necessary for the construction and implementation of the UTMS. The design is applicable to the software's structural, functional, and performance concerns without restriction by size or complexity.

1.3 Context

The UTMS design is situated within the context of a 2-person development team and follows a monolithic deployment architecture. It specifies one or more design languages for each viewpoint, primarily utilizing **UML** and **PlantUML** to ensure well-defined syntax and semantics for all design elements.

1.4 Summary

The SDD is organized into the following design views to address specific stakeholder concerns:

- **Logical Viewpoint:** Details the subsystem decomposition and static class relationships.
- **Information Viewpoint:** Defines the persistent data schema and attribute dictionary.
- **Interaction Viewpoint:** Describes the time-ordered messaging for student submissions and administrative evaluations.
- **Interface Viewpoint:** Establishes the binding contracts and protocols for service consumption.
- **Design Rationale:** Captures the justifications for database and architecture selections.

2. References

The following documents and standards serve as the foundation for the UTMS Software Design Description:

- **IEEE STD 1016-2009:** IEEE Standard for Information Technology—Systems Design—Software Design Descriptions. This standard provides the framework for the viewpoints and organization of this SDD.
- **UTMS Software Requirements Specification (SRS) v8:** The authoritative document defining the functional and non-functional requirements that this design fulfills .
- **ISO/IEC/IEEE 29148:2018:** Systems and software engineering — Life cycle processes — Requirements engineering.
- **IEEE Std 830-1998:** Recommended Practice for Software Requirements Specifications.
- **IZTECH Data-Handling Regulations:** Institutional policies governing the security and privacy of student data.
- **YÖK (Council of Higher Education) Regulations:** National rules defining GPA conversion scales and transfer eligibility.

3. Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability ; a set of properties that guarantee database transactions are processed reliably, essential for academic record integrity.
AES-256	Advanced Encryption Standard with a 256-bit key ; the symmetric encryption algorithm used for data at rest in the UTMS database.
API	Application Programming Interface ; the secure mechanism used for data exchange between UTMS and internal university systems like UBYS.
Dean's Office	The faculty-level administrative office responsible for routing applications to departments and granting final approval for transfer lists.
Intibak	The course-equivalence or credit transfer process where a student's previous academic work is mapped to the IZTECH curriculum.
JSON	JavaScript Object Notation ; the lightweight data-interchange format used for communication between the React frontend and Java backend.
ÖİDB	Öğrenci İşleri Daire Başkanlığı ; the Student Affairs Office responsible for initial application validation and final result publication.
RBAC	Role-Based Access Control ; a security approach that restricts system access to authorized users based on their specific roles (e.g., Student, YGK Member).
REST	Representational State Transfer ; the architectural style used for the web-based communication protocols within UTMS.
SDD	Software Design Description ; this document, which provides the technical blueprint for implementing the UTMS.
UBYS	Üniversite Bilgi Yönetim Sistemi ; the existing University Information System used for retrieving and exchanging student academic records.
YGK	Yatay Geçiş Komisyonu ; the Departmental Transfer Commission responsible for evaluating eligibility and ranking transfer candidates.

Term	Definition
YKS	Yükseköğretim Kurumları Sınavı ; the National University Entrance Exam score used as a primary metric in transfer calculations.

4. Identified Stakeholders and Design Concerns

Each design view in this SDD is organized to address the specific responsibilities and constraints of the actors defined in the system.

Design Stakeholder	Design Concern	Addressed Viewpoint
Student	Secure and efficient submission of personal/academic data and document uploads. Requires clear error feedback and mobile responsiveness.	Logical & Interaction Viewpoints
ÖİDB Officer	Validation integrity and controlled routing. Needs immutable audit logs for forwarding or returning applications.	Information & Interaction Viewpoints
Dean's Office Staff	Faculty-level management, departmental YGK assignment, and final list approval. Requires role-based access control and transparent audit logs.	Logical & Interaction Viewpoints
YGK Member	Automated eligibility checks and precise ranking based on Rule PR-07. Needs fast database query response for evaluation panels.	Information & Interface Viewpoints
System Administrator	System configuration management, RBAC enforcement, and database integrity. Focuses on maintainability.	Logical & Interface Viewpoints

5. Logical Viewpoint

Goal: This viewpoint elaborates on the designed types and their implementations as classes and interfaces with their structural static relationships.

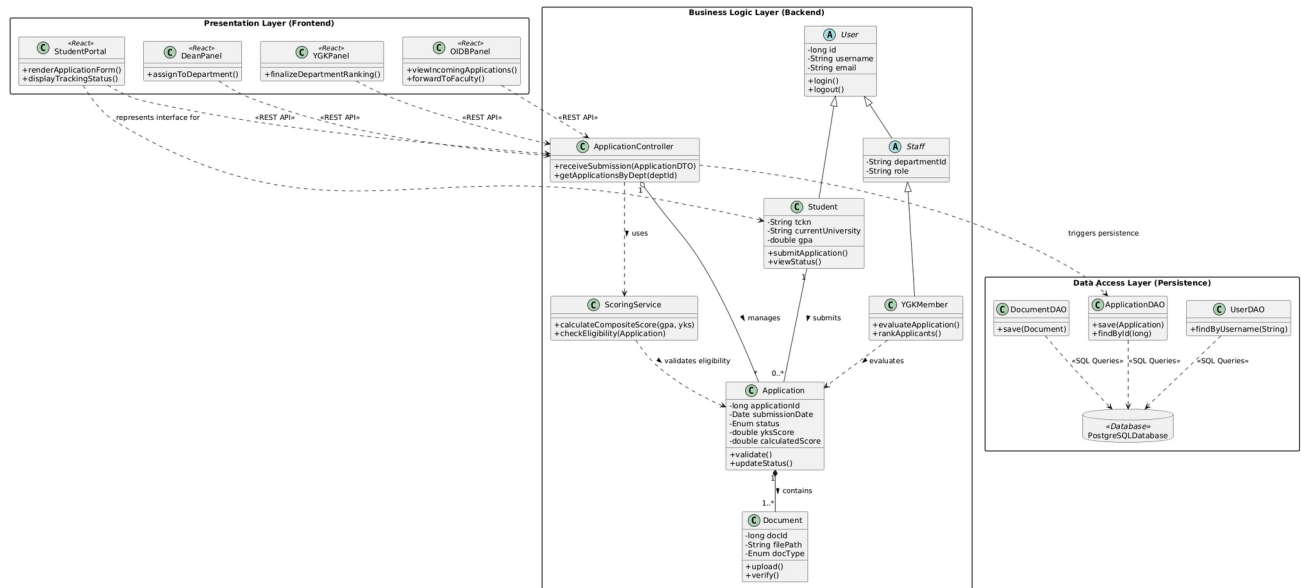
5.1 Subsystem Decomposition

Given the project constraints (2-person team, monolithic deployment), the system follows a layered architecture:

- **Presentation Layer (Frontend):** React-based components responsible for the user interface and communicating with the backend via REST API.
- **Business Logic Layer (Backend):** Java-based Services and Controllers that handle validation, routing, and scoring logic.
- **Data Access Layer (Persistence):** Data Access Objects (DAOs) interacting with the PostgreSQL database.

5.2 Design Class Diagram

The following diagram illustrates the primary Design Entities and their relationships. It details attributes, visibility, and type expressions as required by the standard.

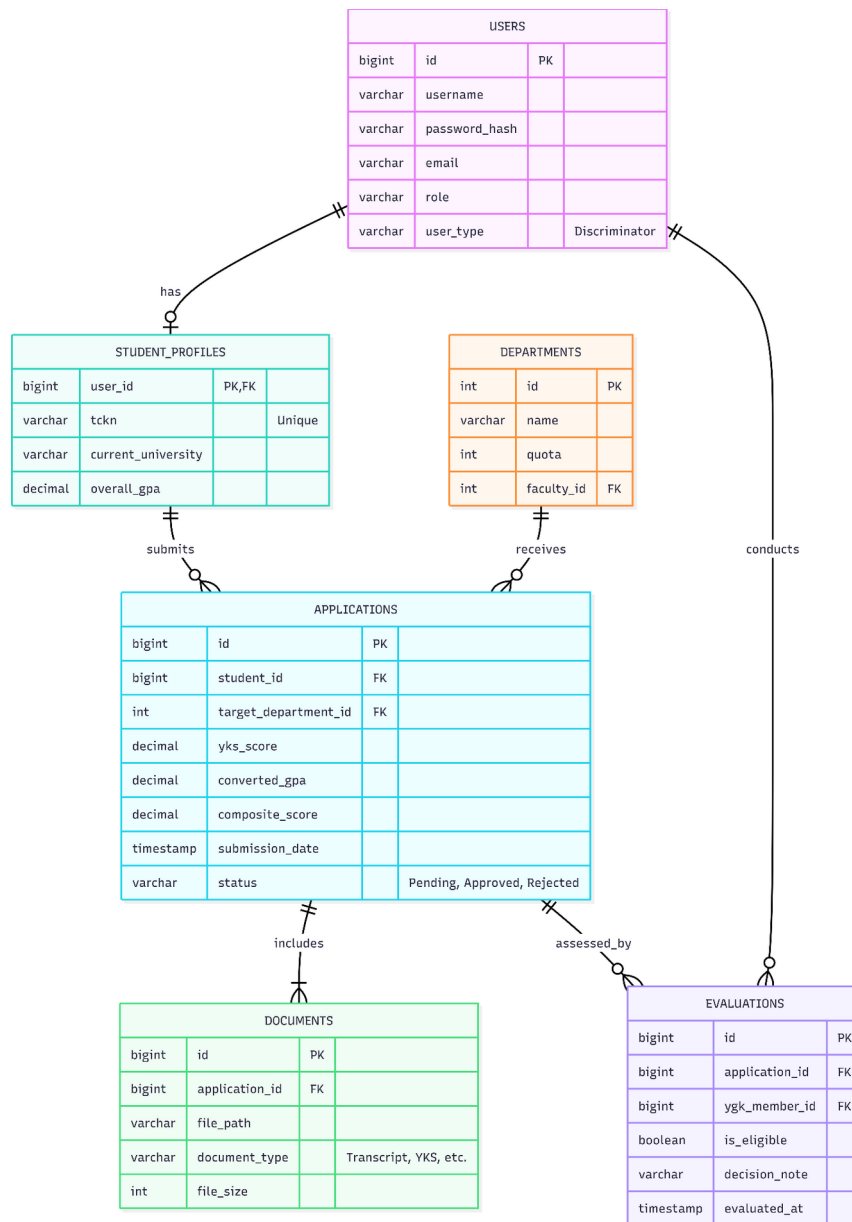


6. Information Viewpoint

Goal: This view addresses persistent information, focusing on the data schema required to support the Functional Requirements.

6.1 Logical Database Schema (ERD)

Based on the SRS Logical Database Requirements and the constraints imposed by PostgreSQL, the following Entity-Relationship Diagram (ERD) defines the schema.



6.2 Data Attribute Dictionary

Specific focus on the Application and Document entities as per SRS requirements:

- **Application Entity:**
 - `yks_score` (Decimal): Stores the National University Entrance Exam score. Critical for ranking.
 - `composite_score` (Decimal): Calculated field based on $\text{Score} = (\text{GPA} * 0.5) + (\text{YKS} * 0.5)$.
 - `Status` (Enum): Tracks lifecycle (New -> Forwarded -> Under Review -> Finalized).
- **Document Entity:**
 - `document_type`: Categorizes files (e.g., Transcript, YKS Result) to enable specific validation rules (e.g., PR-11).
 - `file_path`: Secure reference to the storage location (not the blob itself) to maintain database performance.

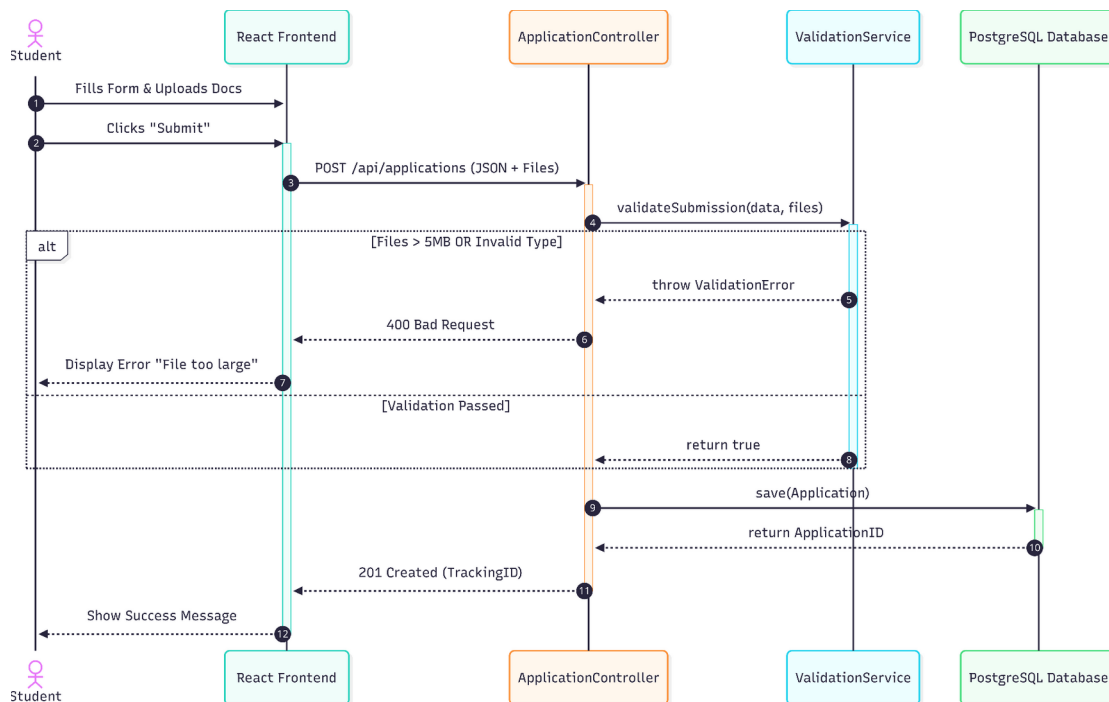
7. Interaction Viewpoint

Goal: Defines strategies for interaction among entities, emphasizing the time ordering of messages.

7.1 Interaction View 1: Student Application Submission

Scope: UC-STU-01 Submit Transfer Application.

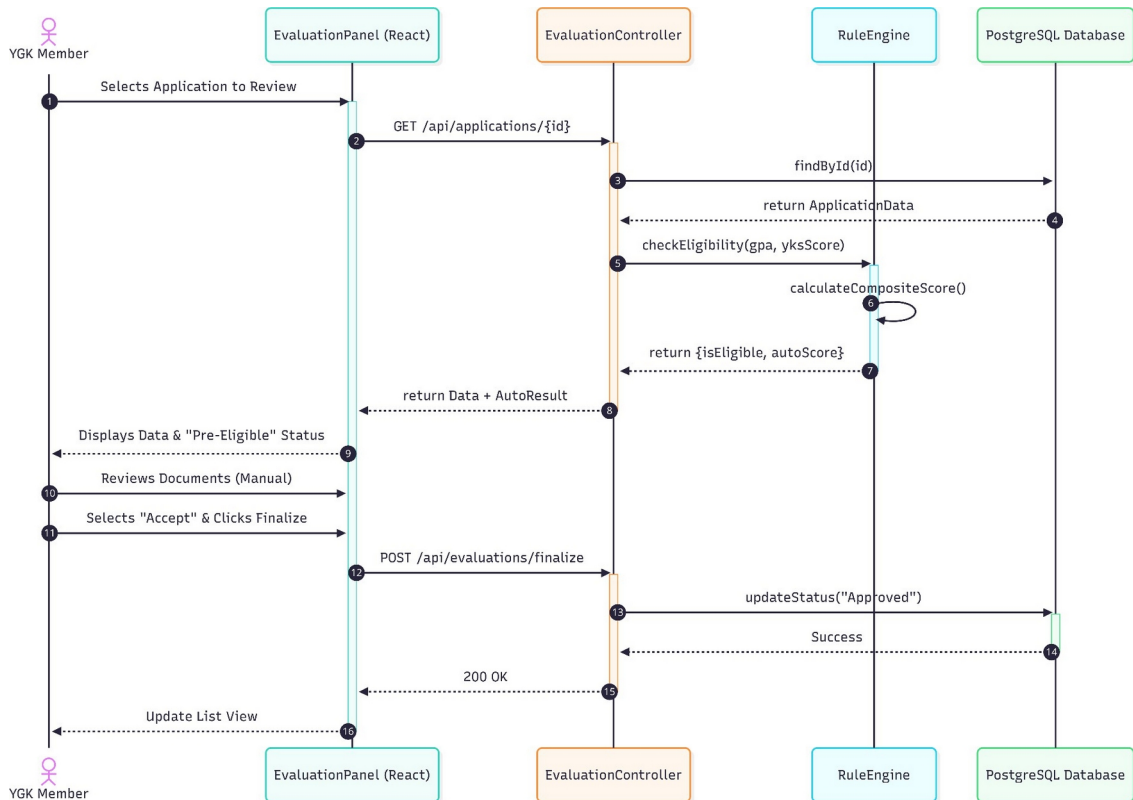
Rationale: This represents the primary data entry point for the system. It demonstrates the interaction between the React Frontend, the Java Controller, and the Persistence layer.



7.2 Interaction View 2: YGK Application Evaluation

Scope: UC-YGK-01 Evaluate Applications.

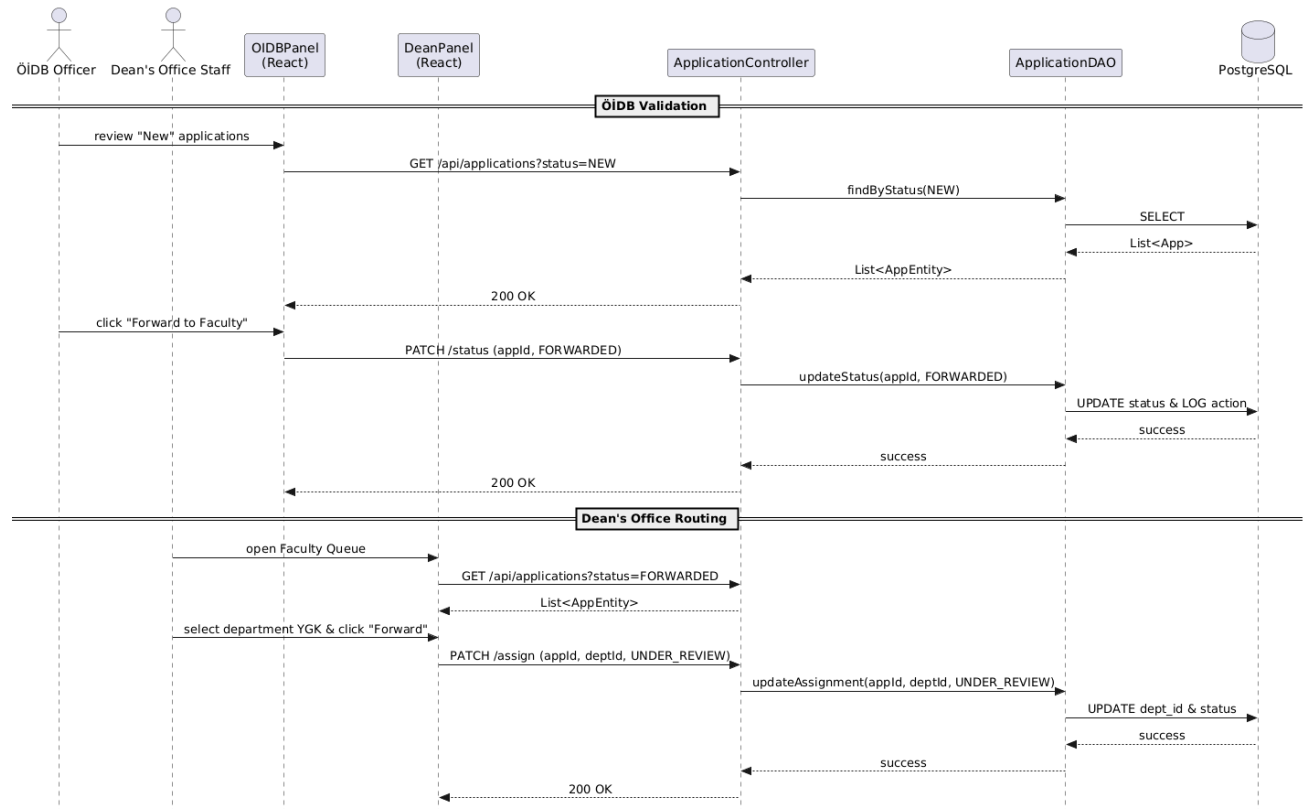
Rationale: This focuses on the administrative workflow, specifically the auto-scoring logic and decision persistence required by the YGK members.



7.3 Interaction View 3: Administrative Validation and Routing

Scope: UC-OIDB-01 Manage and Forward Applications and UC-DEAN-01 Forward to Department Commission

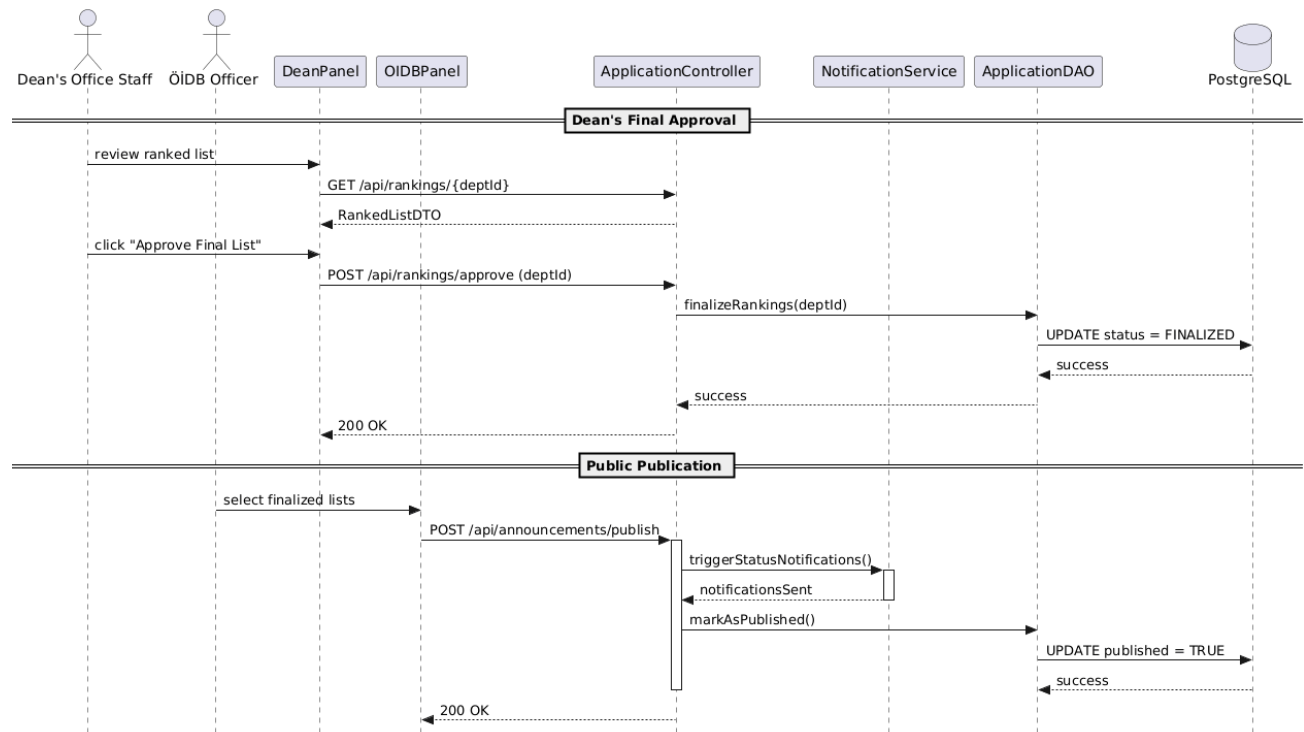
Rationale: This view describes how an ÖIDB Officer validates a submission and how the Dean's Office routes it to the specific department. This represents the critical "gatekeeping" phase of the workflow.



7.4 Interaction View 4: Final Approval and Publication:

Scope: UC-DEAN-02 Approve Final List and UC-OIDB-02 Publish Announcement

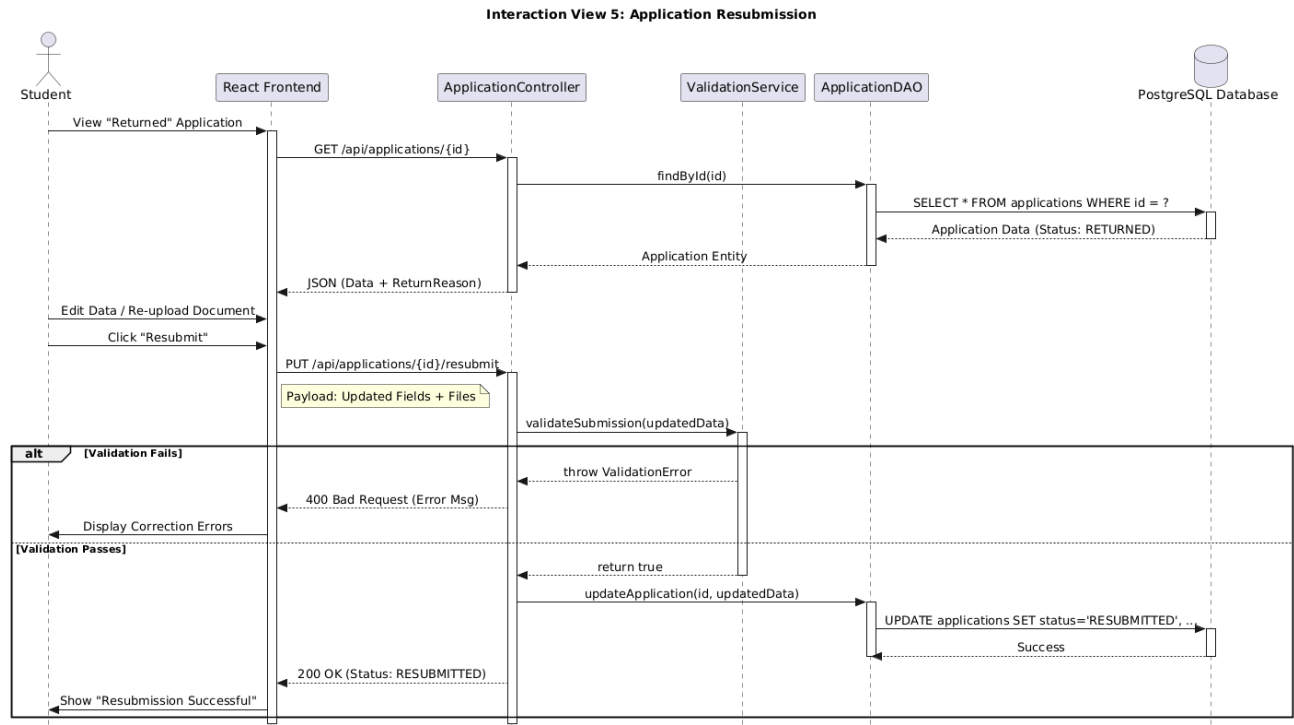
Rationale: This view details the finalization of the cycle, where the Dean's Office approves the ranked lists and ÖİDB publishes them to the public Announcement Module.



7.5 Interaction View 5: Application Resubmission:

Scope: UC-STU-02 Resubmit Returned Application.

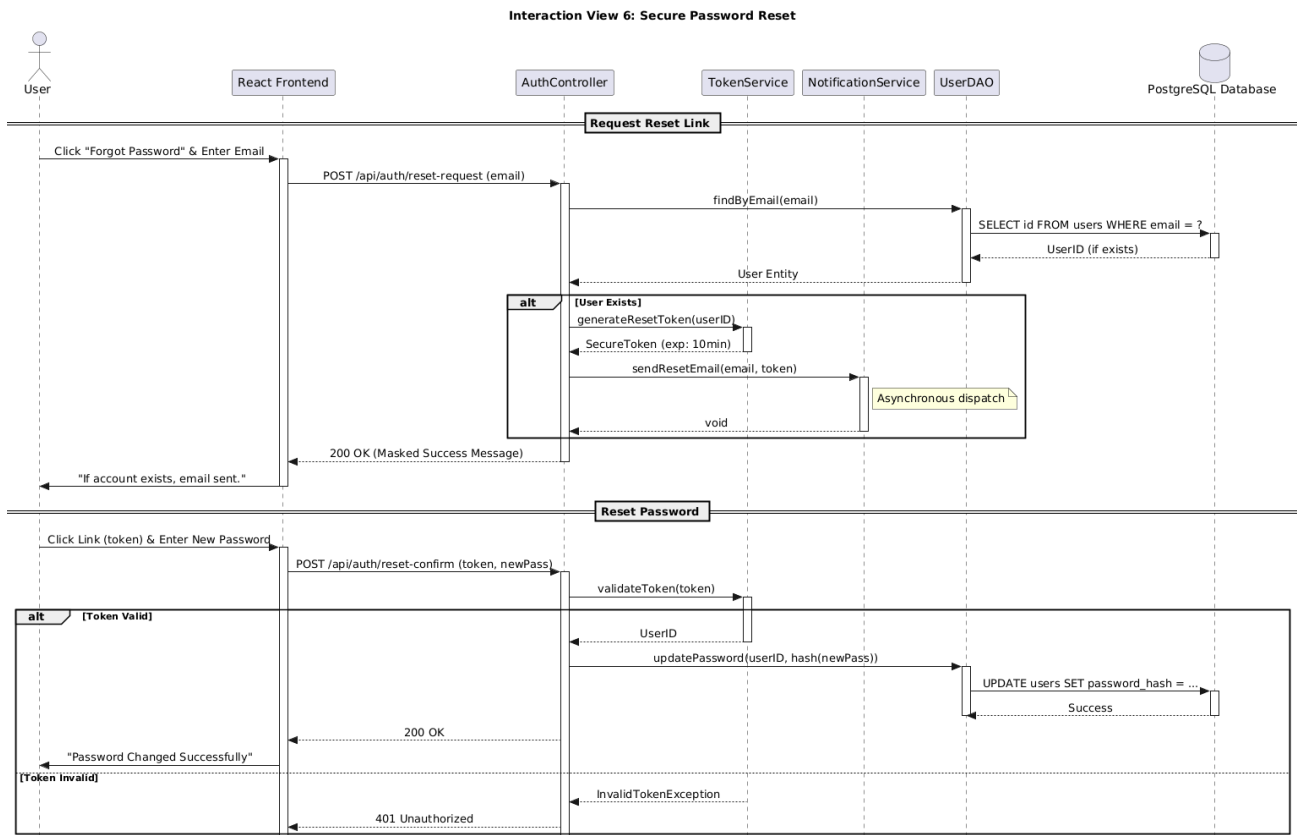
Rationale: This view details the correction workflow. It is critical to show that a resubmitted application undergoes the same rigorous validation logic as a new submission to prevent invalid data from re-entering the processing pipeline. The status transition from RETURNED to RESUBMITTED triggers the workflow to restart at the ÖIDB stage.



7.6 Interaction View 6: Secure Password Reset

Scope: UC-SYS-03 Password Reset.

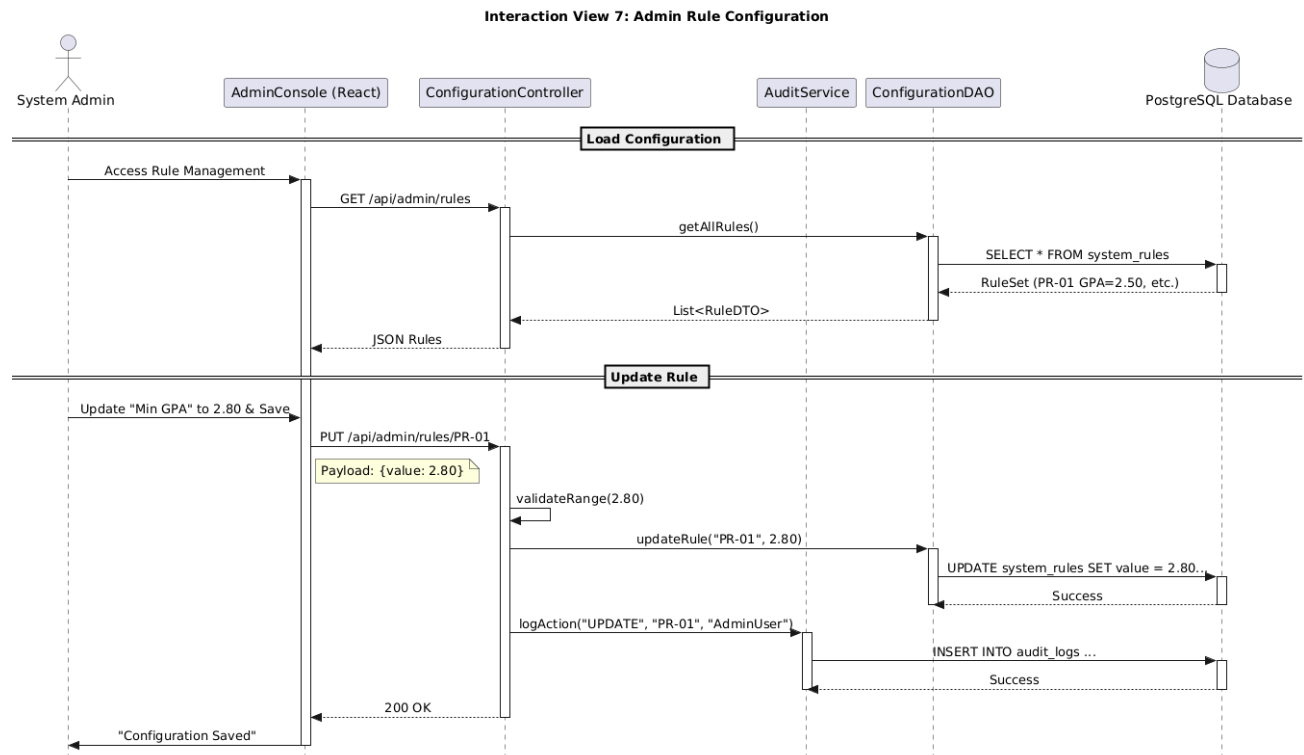
Rationale: This view addresses the security concern of credential recovery. It isolates the sensitive logic of token generation (TokenService) and external communication (NotificationService) from the core user management, ensuring that authentication secrets are never exposed to the client-side React Frontend directly.



7.7 Interaction View 7: Admin Rule Configuration

Scope: UC-ADM-01 Manage Rules & Users.

Rationale: This view handles the dynamic configuration of business rules (e.g., Minimum GPA). Unlike standard CRUD, this interaction emphasizes the "Audit Logging" requirement (FR-20) to maintain system integrity. The `ConfigurationController` acts as the interface for the `AdminConsole`, modifying the persistence layer which the `RuleEngine` (seen in View 2) will query during evaluations.



8 Design Rationale

Goal: Capture the designer's reasoning that led to the system as designed.

8.1 Architectural Choice: A Sturdy, Unified Foundation

We chose a 3-Layered Monolithic Architecture because it provides a clear and organized "home" for every part of the system. For a specialized process like university transfers, this structure ensures that when a student clicks "submit" in the React Frontend, the Java Backend can immediately process it without the risk of data getting lost between disconnected services. It allows our small team to maintain high code quality and ensures the system is easy to update as university policies evolve.

8.2 Data Strategy: Protecting the Academic Record

We selected a relational database (PostgreSQL) instead of a NoSQL solution for the following reasons:

- 1. Structured Data:** The data (Transcripts, GPA, YKS Scores) is highly structured, with strict data types (Decimal, Integer) required by the calculation algorithms.
- 2. Integrity & Compliance:** The system manages official academic records. Relational databases provide ACID (Atomicity, Consistency, Isolation, Durability) guarantees, ensuring that a transfer decision never becomes invalid. This supports the strict "Regulatory Compliance" constraint.
- 3. Relationships:** The data model relies heavily on relationships. SQL Joins are more efficient for generating the complex reports and ranked lists required by the "Approve Final List" use case.

8.3 Data Strategy: Protecting the Academic Record:

We selected a REST API architecture to mediate between the Java Backend and React Frontend:

- 1. Decoupling:** This allows the "Full Stack" team members to work on the Frontend (UI/UX) and Backend independently when needed, adhering to the "Joint/Split by Task" work allocation strategy.
- 2. Statelessness:** REST's stateless nature simplifies server-side logic in the ApplicationController, ensuring scalability as the number of concurrent student users increases during the tight application window.
- 3. Integration:** The UTMS must integrate with the external "UBYS" system. A RESTful approach aligns with modern integration standards used for student record exchange.

8.4 Centralized Logic: Ensuring Rule Consistency

We moved all evaluation and scoring logic into dedicated Service Classes (Scoring and Validation)

- **Equality:** By centralizing the calculation for the Composite Transfer Score ($GPA * 0.5 + (YKS * 0.5)$), we guarantee that every single applicant is judged by the exact same mathematical standard, with no room for human error in the math.
- **Flexibility:** If the Council of Higher Education (YÖK) changes the weight of GPA next year, we only have to change one line of code in the ScoringService to update the entire system.

8.5 Security: Building Trust Through Accountability

Because we handle sensitive personal information like T.C. ID numbers and transcripts, security is not just a feature; it is a promise.

- **Role-Based Access (RBAC):** We ensure that a Dean can approve lists, but cannot accidentally change a student's GPA, and a student can track their status without seeing other applicants' private data.
- **The "Immutable" Log:** Every status change is timestamped and locked. This ensures that if a student asks why their status changed, the ÖİDB can provide a clear, documented answer, fostering trust in the IzTech transfer process.